nsbaci

0.1

Generated by Doxygen 1.15.0

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

nsbaci
Root namespace for the nsbaci application . . . . . ??
nsbaci::services
Services namespace for nsbaci . . . . . ??
nsbaci::types
Type definitions namespace for nsbaci . . . . . ??

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

nsbaci::types::CompileError . . . . . ??
nsbaci::services::CompilerService . . . . . ??
nsbaci::Error . . . . . ??
nsbaci::types::ErrorBase . . . . . ??
nsbaci::ui::ErrorDialogFactory . . . . . ??
FileResult . . . . . ??
    LoadResult . . . . . ??
    saveResult . . . . . ??
nsbaci::services::FileService . . . . . ??
nsbaci::types::LoadError . . . . . ??
QMainWindow
    MainWindow . . . . . ??
QObject
    nsbaci::Controller . . . . . ??
QPlainTextEdit
    CodeEditor . . . . . ??
QWidget
    LineNumberArea . . . . . ??
nsbaci::types::RuntimeError . . . . . ??
nsbaci::services::RuntimeService . . . . . ??
nsbaci::types::SaveError . . . . . ??
nsbaci::UIError . . . . . ??

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

CodeEditor . . . . . ??
nsbaci::types::CompileError
Error payload for compilation errors . . . . . ??
nsbaci::services::CompilerService . . . . . ??
nsbaci::Controller . . . . . ??
nsbaci::Error
Represents an error with a message and optional code . . . . . ??
nsbaci::types::ErrorBase
Base structure containing common error properties . . . . . ??
nsbaci::ui::ErrorDialogFactory
Factory for creating error dialogs from UIError objects . . . . . ??
FileResult . . . . . ??
nsbaci::services::FileService . . . . . ??
LineNumberArea . . . . . ??
nsbaci::types::LoadError
Error payload for file load errors . . . . . ??
LoadResult . . . . . ??
MainWindow . . . . . ??
nsbaci::types::RuntimeError
Error payload for runtime errors . . . . . ??
nsbaci::services::RuntimeService . . . . . ??
nsbaci::types::SaveError
Error payload for file save errors . . . . . ??
saveResult . . . . . ??
nsbaci::UIError
UI-ready error representation for display in dialogs . . . . . ??

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

controller.cpp
Implementation unit for the Controller . . . . . ??
controller.h
Controller class declaration for nsbaci . . . . . ??
error.cpp
Implementation unit for the Error class . . . . . ??
error.h
Error class declaration for nsbaci . . . . . ??
uiError.cpp
Implementation of UIError utilities . . . . . ??
uiError.h
UI Error type definitions for nsbaci . . . . . ??
main.cpp
Application entry point for nsbaci . . . . . ??
compilerService.cpp
Implementation unit for the CompilerService . . . . . ??
compilerService.h
CompilerService class declaration for nsbaci . . . . . ??
fileService.cpp
Implementation unit for the FileService . . . . . ??
fileService.h
FileService class declaration for nsbaci . . . . . ??
runtimeService.cpp
Implementation unit for the RuntimeService . . . . . ??
runtimeService.h
RuntimeService class declaration for nsbaci . . . . . ??
compilerTypes.h
Type definitions for compiler-related operations . . . . . ??
errorTypes.h
Type definitions for error-related structures . . . . . ??
fileTypes.h
Type definitions for file-related operations . . . . . ??
runtimeTypes.h
Type definitions for runtime-related operations . . . . . ??
errorDialogFactory.cpp
Implementation of ErrorDialogFactory . . . . . ??

errorDialogFactory.h
Factory for creating error dialogs from UIError objects . . . . . ??
codeeditor.cpp
Implementation of the CodeEditor class with line numbers . . . . . ??
codeeditor.h
CodeEditor class with line numbers for nsbaci . . . . . ??
mainwindow.cpp
Implementation of the MainWindow class . . . . . ??
mainwindow.h
Main window class declaration for nsbaci . . . . . ??

Chapter 5

Namespace Documentation

5.1 nsbaci Namespace Reference

Root namespace for the nsbaci application.

Namespaces

- namespace services

  Services namespace for nsbaci.
- namespace types

  Type definitions namespace for nsbaci.

Classes

- class Controller
- class Error

  Represents an error with a message and optional code.
- struct UIError

  UI-ready error representation for display in dialogs.

5.1.1 Detailed Description

Root namespace for the nsbaci application.

5.2 nsbaci::services Namespace Reference

Services namespace for nsbaci.

Classes

- class CompilerService
- class FileService
- class RuntimeService

5.2.1 Detailed Description

Services namespace for nsbaci.

5.3 nsbaci::types Namespace Reference

Type definitions namespace for nsbaci.

Classes

- struct ErrorBase

  Base structure containing common error properties.
- struct CompileError

  Error payload for compilation errors.
- struct SaveError

  Error payload for file save errors.
- struct LoadError

  Error payload for file load errors.
- struct RuntimeError

  Error payload for runtime errors.

Typedefs

- using **ErrMessage** = std::string
- using ErrorPayload

  Variant type for all possible error payloads.
- using **Text** = std::string

  Alias for text content (file contents, source code, etc.).
- using **File** = fs::path

  Alias for file system paths.

Enumerations

- enum class ErrSeverity { **Warning** , **Error** , **Fatal** }

  Severity levels for errors.
- enum class ErrType {
  emptyPath , **invalidPath** , **invalidExtension** , **directoryNotFound** ,
  fileNotFound , **notARegularFile** , **permissionDenied** , **openFailed** ,
  readFailed , **writeFailed** , **unknown** }

  Types of errors that can occur in the application.

5.3.1 Detailed Description

Type definitions namespace for nsbaci.

5.3.2 Typedef Documentation

5.3.2.1 ErrorPayload

```
using nsbaci::types::ErrorPayload
```

Initial value:

```
std::variant<SaveError, LoadError, CompileError, RuntimeError>
```

Variant type for all possible error payloads.

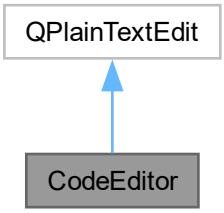
Can be used to create a more specific error message in the controller, for example when an error is of type compile↩ Error, the controller can specify the line, col, what might have happened...

Chapter 6

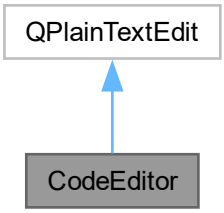
Class Documentation

6.1 CodeEditor Class Reference

Inheritance diagram for CodeEditor:



Collaboration diagram for CodeEditor:



Public Member Functions

- **CodeEditor** (QWidget *parent=nullptr)
- void **lineNumberAreaPaintEvent** (QPaintEvent *event)
- int **lineNumberAreaWidth** ()

Protected Member Functions

- void **resizeEvent** (QResizeEvent *event) override

The documentation for this class was generated from the following files:

- codeeditor.h
- codeeditor.cpp

6.2 nsbaci::types::CompileError Struct Reference

Error payload for compilation errors.

```
#include <errorTypes.h>
```

6.2.1 Detailed Description

Error payload for compilation errors.

The documentation for this struct was generated from the following file:

- errorTypes.h

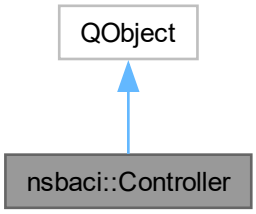
6.3 nsbaci::services::CompilerService Class Reference

The documentation for this class was generated from the following file:

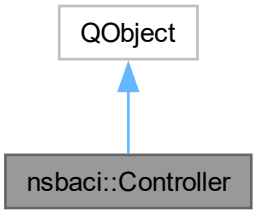
- compilerService.h

6.4 nsbaci::Controller Class Reference

Inheritance diagram for nsbaci::Controller:



Collaboration diagram for nsbaci::Controller:



Public Slots

- void **onSaveRequested** (File file, Text contents)
- void **onOpenRequested** (File file)
- void **onCompileRequested** (Text contents)
- void **onRunRequested** ()

Signals

- void **saveFailed** (std::vector< UIError > errors)
- void **saveSucceeded** ()
- void **loadFailed** (std::vector< UIError > errors)
- void **loadSucceeded** (const QString &contents)

Public Member Functions

- **Controller** (QObject *parent=nullptr)

The documentation for this class was generated from the following files:

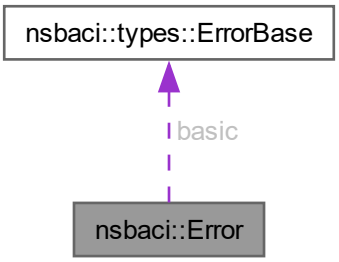
- controller.h
- controller.cpp

6.5 nsbaci::Error Class Reference

Represents an error with a message and optional code.

```
#include <error.h>
```

Collaboration diagram for nsbaci::Error:



Public Attributes

- ErrorBase **basic**
- ErrorPayload **payload**

6.5.1 Detailed Description

Represents an error with a message and optional code.

The documentation for this class was generated from the following file:

- error.h

6.6 nsbaci::types::ErrorBase Struct Reference

Base structure containing common error properties.

```
#include <errorTypes.h>
```

Public Attributes

- ErrSeverity **severity**
- ErrMessage **message**
- ErrType **type**

6.6.1 Detailed Description

Base structure containing common error properties.

The documentation for this struct was generated from the following file:

- errorTypes.h

6.7 nsbaci::ui::ErrorDialogFactory Class Reference

Factory for creating error dialogs from UIError objects.

```
#include <errorDialogFactory.h>
```

Public Types

- using DialogInvoker = std::function<QMessageBox::StandardButton()>

  Callable type that shows a dialog when invoked.

Static Public Member Functions

- static DialogInvoker getDialogFromUIError (const UIError &error, QWidget *parent=nullptr)

  Creates a dialog invoker from a UIError.
- static std::vector< DialogInvoker > getDialogsFromUIErrors (const std::vector< UIError > &errors, QWidget *parent=nullptr)

  Creates dialog invokers for multiple UIErrors.
- static DialogInvoker getSuccessDialog (const QString &title, const QString &message, QWidget *parent=nullptr)

  Creates a success message dialog invoker.
- static void showErrors (const std::vector< UIError > &errors, QWidget *parent=nullptr)

  Shows all error dialogs sequentially.
- static QMessageBox::StandardButton showError (const UIError &error, QWidget *parent=nullptr)

  Shows a single error dialog immediately.
- static void showSuccess (const QString &title, const QString &message, QWidget *parent=nullptr)

  Shows a success message dialog immediately.

6.7.1 Detailed Description

Factory for creating error dialogs from UIError objects.

Provides static methods to convert UIError objects into QMessageBox dialogs. Supports two modes:

- **Deferred**: Returns a DialogInvoker callable for later invocation
- **Immediate**: Shows the dialog right away via convenience methods

See also

DialogInvoker

6.7.2 Member Typedef Documentation

6.7.2.1 DialogInvoker

```
using nsbaci::ui::ErrorDialogFactory::DialogInvoker = std::function<QMessageBox::Standard↩
Button()>
```

Callable type that shows a dialog when invoked.

A DialogInvoker is a packaged dialog that can be invoked at any time. When called, it displays the dialog (blocking) and returns which button the user clicked. Think of it as a "suspended dialog" or "dialog builder" that you can trigger when ready.

Returns

The button that was clicked (QMessageBox::StandardButton).

6.7.3 Member Function Documentation

6.7.3.1 getDialogFromUIError()

```
ErrorDialogFactory::DialogInvoker nsbaci::ui::ErrorDialogFactory::getDialogFromUIError (
            const UIError & error,
            QWidget * parent = nullptr)  [static]
```

Creates a dialog invoker from a UIError.

Returns a callable that, when invoked, shows the error dialog and returns the button clicked by the user.

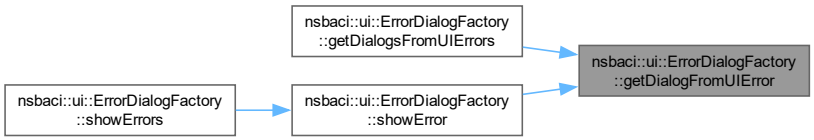
Parameters

error	The UIError to display.
parent	Parent widget for the dialog.

Returns

A callable that shows the dialog when invoked.

Here is the caller graph for this function:



6.7.3.2 getDialogsFromUIErrors()

```
std::vector< ErrorDialogFactory::DialogInvoker > nsbaci::ui::ErrorDialogFactory::getDialogs↩
FromUIErrors (
            const std::vector< UIError > & errors,
            QWidget * parent = nullptr)  [static]
```

Creates dialog invokers for multiple UIErrors.

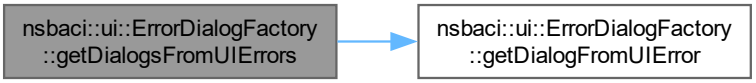
Parameters

errors	Vector of UIErrors to convert.
parent	Parent widget for all dialogs.

Returns

Vector of callables, one per error.

Here is the call graph for this function:



6.7.3.3 getSuccessDialog()

```
ErrorDialogFactory::DialogInvoker nsbaci::ui::ErrorDialogFactory::getSuccessDialog (
            const QString & title,
            const QString & message,
            QWidget * parent = nullptr)  [static]
```

Creates a success message dialog invoker.

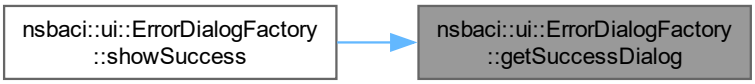
Parameters

title	Dialog title.
message	Success message body.
parent	Parent widget for the dialog.

Returns

A callable that shows the success dialog when invoked.

Here is the caller graph for this function:



6.7.3.4 showError()

```
QMessageBox::StandardButton nsbaci::ui::ErrorDialogFactory::showError (
            const UIError & error,
            QWidget * parent = nullptr)  [static]
```

Shows a single error dialog immediately.

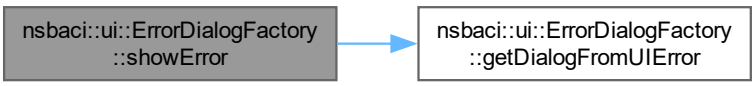
Parameters

error	The UIError to display.
parent	Parent widget for the dialog.

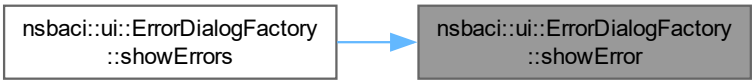
Returns

The button that was clicked.

Here is the call graph for this function:



Here is the caller graph for this function:



6.7.3.5 showErrors()

```
void nsbaci::ui::ErrorDialogFactory::showErrors (
            const std::vector< UIError > & errors,
            QWidget * parent = nullptr)  [static]
```

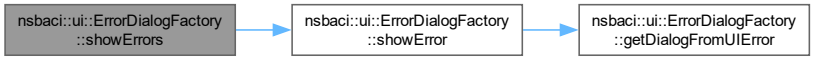
Shows all error dialogs sequentially.

Convenience method that creates and immediately shows dialogs for all provided UIErrors.

Parameters

errors	Vector of UIErrors to display.
parent	Parent widget for all dialogs.

Here is the call graph for this function:



6.7.3.6 showSuccess()

```
void nsbaci::ui::ErrorDialogFactory::showSuccess (
            const QString & title,
            const QString & message,
            QWidget * parent = nullptr)  [static]
```

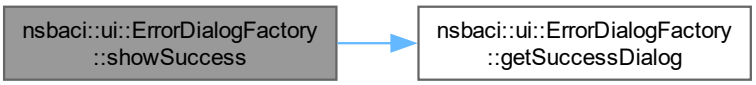
Shows a success message dialog immediately.

Parameters

title	Dialog title.

message	Success message body.
parent	Parent widget for the dialog.

Here is the call graph for this function:

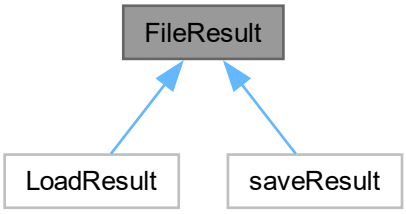


The documentation for this class was generated from the following files:

- errorDialogFactory.h
- errorDialogFactory.cpp

6.8 FileResult Struct Reference

Inheritance diagram for FileResult:



Public Member Functions

- **FileResult** (std::vector< Error > errs)
- **FileResult** (Error error)
- **FileResult** (FileResult &&) noexcept=default
- FileResult & **operator=** (FileResult &&) noexcept=default
- **FileResult** (const FileResult &)=default
- FileResult & **operator=** (const FileResult &)=default

Public Attributes

- bool **ok**
- std::vector< Error > **errors**

The documentation for this struct was generated from the following file:

- fileService.h

6.9 nsbaci::services::FileService Class Reference

Public Member Functions

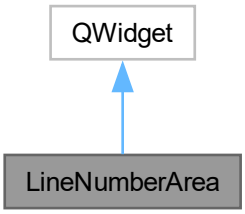
- saveResult **save** (Text contents, File file)
- LoadResult **load** (File file)

The documentation for this class was generated from the following files:

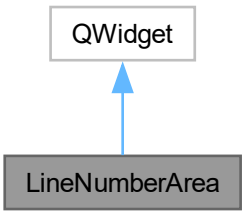
- fileService.h
- fileService.cpp

6.10 LineNumberArea Class Reference

Inheritance diagram for LineNumberArea:



Collaboration diagram for LineNumberArea:



Public Member Functions

- **LineNumberArea** (CodeEditor *editor)
- QSize **sizeHint** () const override

Protected Member Functions

- void **paintEvent** (QPaintEvent *event) override

The documentation for this class was generated from the following file:

- codeeditor.h

6.11 nsbaci::types::LoadError Struct Reference

Error payload for file load errors.

```
#include <errorTypes.h>
```

Public Attributes

- File **associatedFile**

6.11.1 Detailed Description

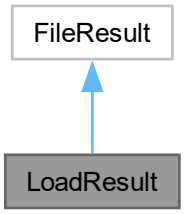
Error payload for file load errors.

The documentation for this struct was generated from the following file:

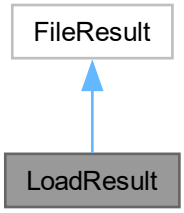
- errorTypes.h

6.12 LoadResult Struct Reference

Inheritance diagram for LoadResult:



Collaboration diagram for LoadResult:



Public Member Functions

- **LoadResult** (Text conts, File name)
- **LoadResult** (std::vector< Error > errs)
- **LoadResult** (Error error)
- **LoadResult** (LoadResult &&) noexcept=default
- LoadResult & **operator=** (LoadResult &&) noexcept=default
- **LoadResult** (const LoadResult &)=default
- LoadResult & **operator=** (const LoadResult &)=default

Public Member Functions inherited from FileResult

- **FileResult** (std::vector< Error > errs)
- **FileResult** (Error error)
- **FileResult** (FileResult &&) noexcept=default
- FileResult & **operator=** (FileResult &&) noexcept=default
- **FileResult** (const FileResult &)=default
- FileResult & **operator=** (const FileResult &)=default

Public Attributes

- Text **contents**
- File **fileName**

Public Attributes inherited from FileResult

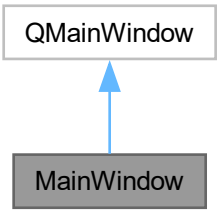
- bool **ok**
- std::vector< Error > **errors**

The documentation for this struct was generated from the following file:

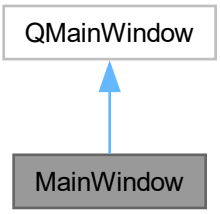
- fileService.h

6.13 MainWindow Class Reference

Inheritance diagram for MainWindow:



Collaboration diagram for MainWindow:



Public Slots

- void **setEditorContents** (const QString &contents)
- void **setStatusMessage** (const QString &message)
- void **setCurrentFile** (const QString &fileName, bool modified=false)
- void **onSaveSucceeded** ()
- void **onSaveFailed** (std::vector< nsbaci::UIError > errors)
- void **onLoadSucceeded** (const QString &contents)
- void **onLoadFailed** (std::vector< nsbaci::UIError > errors)

Signals

- void **saveRequested** (const QString &filePath, const QString &contents)
- void **openRequested** (const QString &filePath)
- void **compileRequested** (const QString &contents)
- void **runRequested** ()

Public Member Functions

- **MainWindow** (QWidget *parent=nullptr)

The documentation for this class was generated from the following files:

- mainwindow.h
- mainwindow.cpp

6.14 nsbaci::types::RuntimeError Struct Reference

Error payload for runtime errors.

```
#include <errorTypes.h>
```

6.14.1 Detailed Description

Error payload for runtime errors.

The documentation for this struct was generated from the following file:

- errorTypes.h

6.15 nsbaci::services::RuntimeService Class Reference

The documentation for this class was generated from the following file:

- runtimeService.h

6.16 nsbaci::types::SaveError Struct Reference

Error payload for file save errors.

```
#include <errorTypes.h>
```

Public Attributes

- File **associatedFile**

6.16.1 Detailed Description

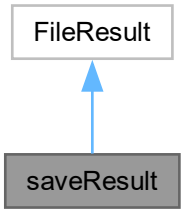
Error payload for file save errors.

The documentation for this struct was generated from the following file:

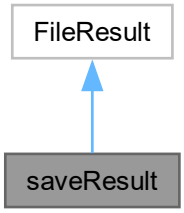
- errorTypes.h

6.17 saveResult Struct Reference

Inheritance diagram for saveResult:



Collaboration diagram for saveResult:



Public Member Functions

- **saveResult** (std::vector< Error > errs)
- **saveResult** (Error error)
- **saveResult** (saveResult &&) noexcept=default
- saveResult & **operator=** (saveResult &&) noexcept=default
- **saveResult** (const saveResult &)=default
- saveResult & **operator=** (const saveResult &)=default

Public Member Functions inherited from FileResult

- **FileResult** (std::vector< Error > errs)
- **FileResult** (Error error)
- **FileResult** (FileResult &&) noexcept=default
- FileResult & **operator=** (FileResult &&) noexcept=default
- **FileResult** (const FileResult &)=default
- FileResult & **operator=** (const FileResult &)=default

Additional Inherited Members

Public Attributes inherited from FileResult

- bool **ok**
- std::vector< Error > **errors**

The documentation for this struct was generated from the following file:

- fileService.h

6.18 nsbaci::UIError Struct Reference

UI-ready error representation for display in dialogs.

```
#include <uiError.h>
```

Static Public Member Functions

- static std::vector< UIError > fromBackendErrors (const std::vector< Error > &errors)

  Converts backend errors to UI-ready errors.

Public Attributes

- QString **title**
- QString **body**
- ErrSeverity **severity**

6.18.1 Detailed Description

UI-ready error representation for display in dialogs.

Contains all the information needed to render an error dialog: title, body text, and severity (which maps to an icon).

6.18.2 Member Function Documentation

6.18.2.1 fromBackendErrors()

```
std::vector< UIError > nsbaci::UIError::fromBackendErrors (
            const std::vector< Error > & errors)  [static]
```

Converts backend errors to UI-ready errors.

Parameters

errors	Vector of backend Error objects.

Returns

Vector of UIError objects ready for display.

The documentation for this struct was generated from the following files:

- uiError.h
- uiError.cpp

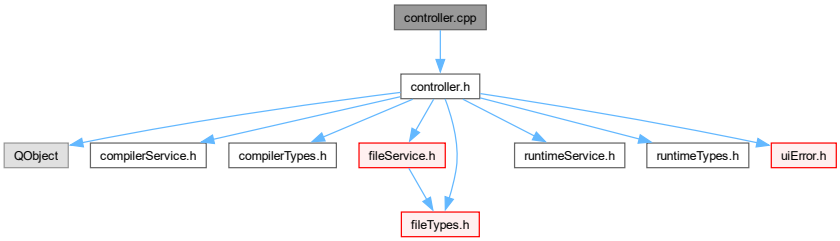
Chapter 7

File Documentation

7.1 controller.cpp File Reference

Implementation unit for the Controller.

Include dependency graph for controller.cpp:



Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.

7.1.1 Detailed Description

Implementation unit for the Controller.

Author

Nicolás Serrano García

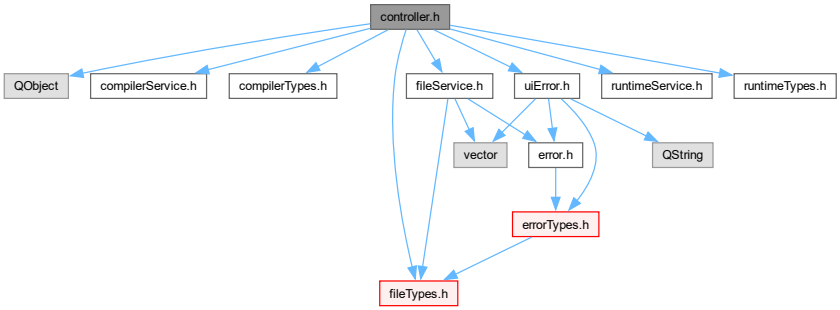
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

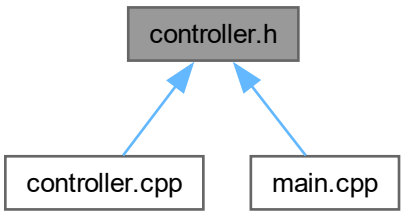
7.2 controller.h File Reference

Controller class declaration for nsbaci.

Include dependency graph for controller.h:



This graph shows which files directly or indirectly include this file:



Classes

- class nsbaci::Controller

Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.

7.2.1 Detailed Description

Controller class declaration for nsbaci.

This module defines the control layer placeholder. The Controller class is intended to coordinate the UI and services.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.3 controller.h

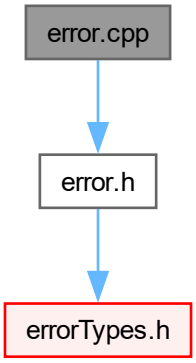
Go to the documentation of this file.

```
00001
00012
00013 #ifndef NSBACI_CONTROLLER_H
00014 #define NSBACI_CONTROLLER_H
00015
00016 #include <QObject>
00017
00018 #include "compilerService.h"
00019 #include "compilerTypes.h"
00020 #include "fileService.h"
00021 #include "fileTypes.h"
00022 #include "runtimeService.h"
00023 #include "runtimeTypes.h"
00024 #include "uiError.h"
00025
00026 using namespace nsbaci::services;
00027 using namespace nsbaci::types;
00028
00033 namespace nsbaci {
00034
00038 class Controller : public QObject {
00039   Q_OBJECT
00040
00041  public:
00042   explicit Controller(QObject* parent = nullptr);
00043   ~Controller() = default;
00044
00045  signals:
00046   // File operation results
00047   void saveFailed(std::vector<UIError> errors);
00048   void saveSucceeded();
00049   void loadFailed(std::vector<UIError> errors);
00050   void loadSucceeded(const QString& contents);
00051
00052  public slots:
00053   // File operations
00054   void onSaveRequested(File file, Text contents);
00055   void onOpenRequested(File file);
00056
00057   // Build operations
00058   void onCompileRequested(Text contents);
00059   void onRunRequested();
00060
00061  private:
00062   FileService fileService;
00063   CompilerService compilerService;
00064   RuntimeService runtimeService;
00065 };
00066
00067 }  // namespace nsbaci
00068
00069 #endif  // NSBACI_CONTROLLER_H
```

7.4 error.cpp File Reference

Implementation unit for the Error class.

Include dependency graph for error.cpp:



Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.

7.4.1 Detailed Description

Implementation unit for the Error class.

Author

Nicolás Serrano García

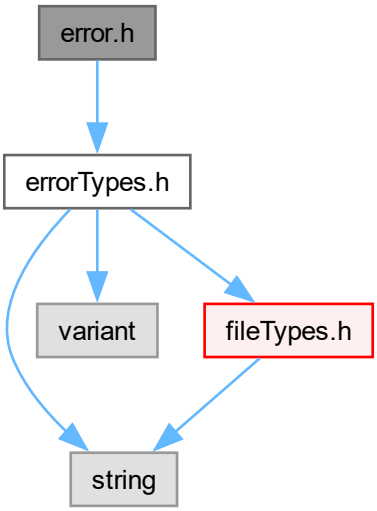
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

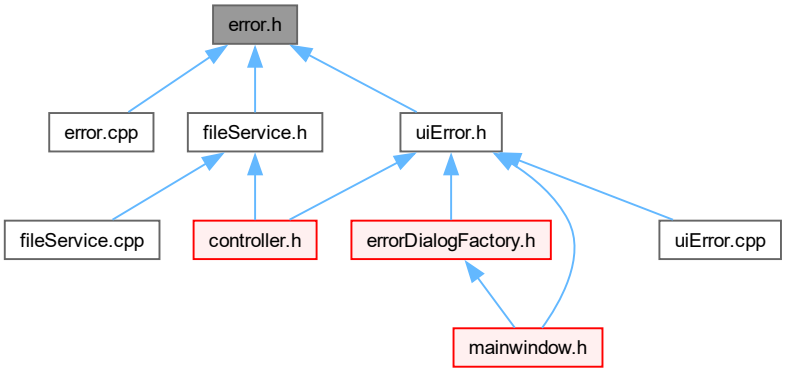
7.5 error.h File Reference

Error class declaration for nsbaci.

Include dependency graph for error.h:



This graph shows which files directly or indirectly include this file:



Classes

- class nsbaci::Error

  Represents an error with a message and optional code.

Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.

7.5.1 Detailed Description

Error class declaration for nsbaci.

This module defines the error handling structures and classes used throughout the application for consistent error reporting.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.6 error.h

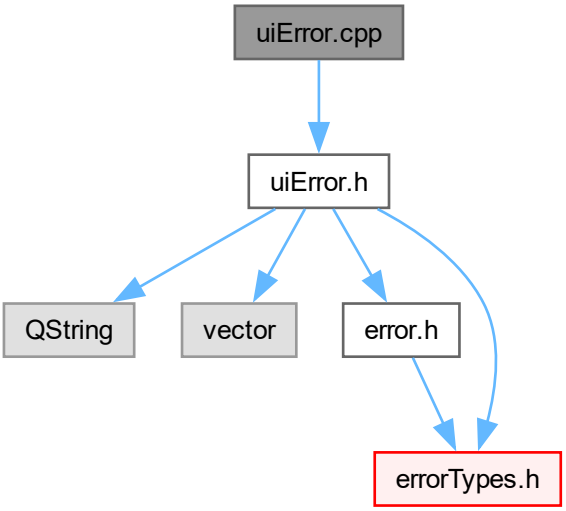
Go to the documentation of this file.

```
00001
00012
00013 #ifndef NSBACI_ERROR_H
00014 #define NSBACI_ERROR_H
00015
00016 #include "errorTypes.h"
00017
00018 using namespace nsbaci::types;
00019
00024 namespace nsbaci {
00025
00030 class Error {
00031  public:
00032   Error() = default;
00033   ~Error() = default;
00034   ErrorBase basic;
00035   ErrorPayload payload;
00036 };
00037
00038 }  // namespace nsbaci
00039
00040 #endif  // NSBACI_ERROR_H
```

7.7 uiError.cpp File Reference

Implementation of UIError utilities.

Include dependency graph for uiError.cpp:



Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.

7.7.1 Detailed Description

Implementation of UIError utilities.

Author

Nicolás Serrano García

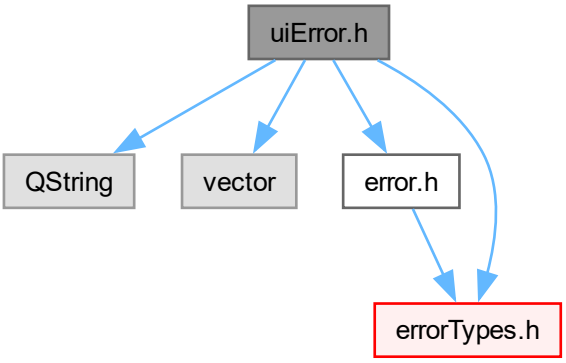
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

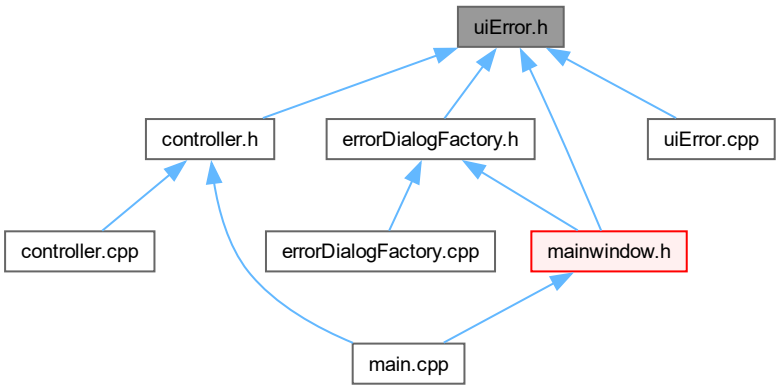
7.8 uiError.h File Reference

UI Error type definitions for nsbaci.

Include dependency graph for uiError.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct nsbaci::UIError

  UI-ready error representation for display in dialogs.

Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.

7.8.1 Detailed Description

UI Error type definitions for nsbaci.

This header provides types used to communicate between the controller and the view layer, particularly for error display.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.9 uiError.h

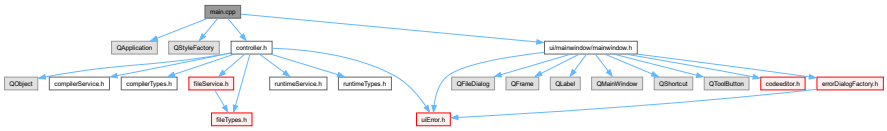
Go to the documentation of this file.

```
00001
00012
00013 #ifndef NSBACI_UIERROR_H
00014 #define NSBACI_UIERROR_H
00015
00016 #include <QString>
00017 #include <vector>
00018
00019 #include "error.h"
00020 #include "errorTypes.h"
00021
00022 using namespace nsbaci;
00023 using namespace nsbaci::types;
00024
00025 namespace nsbaci {
00026
00034 struct UIError {
00035   QString title;         // Dialog title (e.g., "Save Error", "Load Error")
00036   QString body;          // Main message body with details
00037   ErrSeverity severity;  // Maps to icon (Warning, Error, Fatal)
00038
00044   static std::vector<UIError> fromBackendErrors(
00045       const std::vector<Error>& errors);
00046
00047  private:
00051   static QString reasonFromType(ErrType type);
00052
00056   static QString titleFromPayload(const ErrorPayload& payload);
00057
00061   static QString contextFromPayload(const ErrorPayload& payload);
00062 };
00063
00064 }  // namespace nsbaci
00065
00066 #endif  // NSBACI_UIERROR_H
```

7.10 main.cpp File Reference

Application entry point for nsbaci.

Include dependency graph for main.cpp:



Functions

- void **setupViewController** (nsbaci::Controller ∗c, MainWindow ∗w)
- int **main** (int argc, char ∗argv[ ])

7.10.1 Detailed Description

Application entry point for nsbaci.

This file contains the main function that initializes the Qt application and displays the main window.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

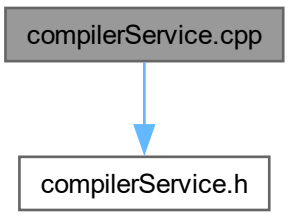
See also

MainWindow

7.11 compilerService.cpp File Reference

Implementation unit for the CompilerService.

Include dependency graph for compilerService.cpp:



Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.
- namespace nsbaci::services

  Services namespace for nsbaci.

7.11.1 Detailed Description

Implementation unit for the CompilerService.

Author

Nicolás Serrano García

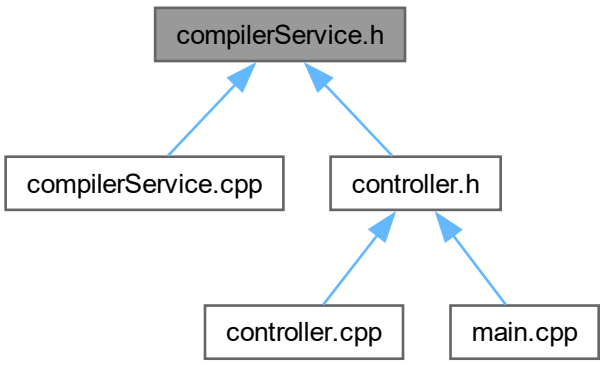
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.12 compilerService.h File Reference

CompilerService class declaration for nsbaci.

This graph shows which files directly or indirectly include this file:



Classes

- class nsbaci::services::CompilerService

Namespaces

- namespace nsbaci::services

  Services namespace for nsbaci.
- namespace nsbaci

  Root namespace for the nsbaci application.

7.12.1 Detailed Description

CompilerService class declaration for nsbaci.

This service is intended to encapsulate compilation-related logic.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.13 compilerService.h

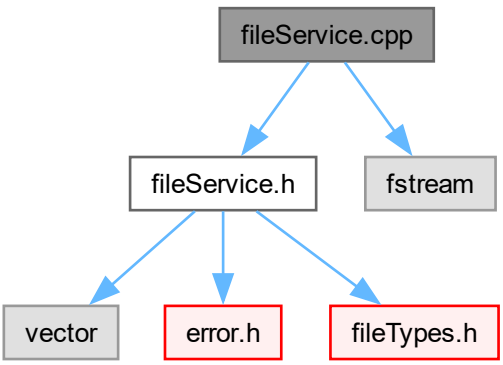
Go to the documentation of this file.

```
00001
00011
00012 #ifndef NSBACI_COMPILERSERVICE_H
00013 #define NSBACI_COMPILERSERVICE_H
00014
00019 namespace nsbaci::services {
00020
00024 class CompilerService {
00025  public:
00026   CompilerService() = default;
00027   ~CompilerService() = default;
00028 };
00029
00030 }  // namespace nsbaci::services
00031
00032 #endif  // NSBACI_COMPILERSERVICE_H
```

7.14 fileService.cpp File Reference

Implementation unit for the FileService.

Include dependency graph for fileService.cpp:



Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.
- namespace nsbaci::services

  Services namespace for nsbaci.

7.14.1 Detailed Description

Implementation unit for the FileService.

Author

Nicolás Serrano García

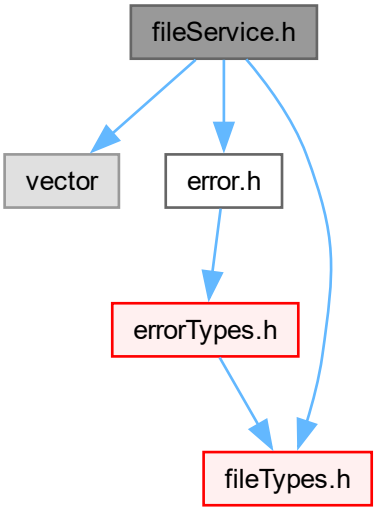
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

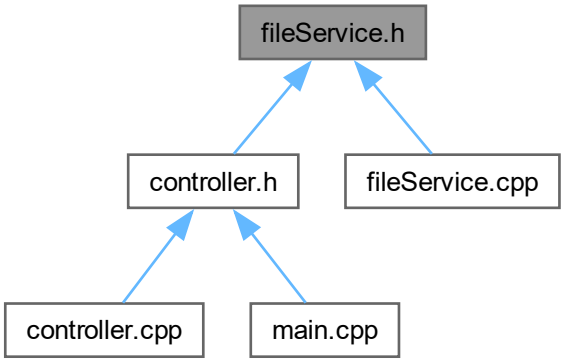
7.15 fileService.h File Reference

FileService class declaration for nsbaci.

Include dependency graph for fileService.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct FileResult
- struct saveResult
- struct LoadResult
- class nsbaci::services::FileService

Namespaces

- namespace nsbaci::services

  Services namespace for nsbaci.
- namespace nsbaci

  Root namespace for the nsbaci application.

7.15.1 Detailed Description

FileService class declaration for nsbaci.

This service is intended to encapsulate file system operations.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.16 fileService.h

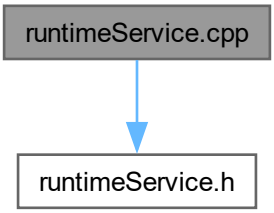
Go to the documentation of this file.

```
00001
00011
00012 #ifndef NSBACI_FILESERVICE_H
00013 #define NSBACI_FILESERVICE_H
00014
00015 #include <vector>
00016
00017 #include "error.h"
00018 #include "fileTypes.h"
00019
00020 using namespace nsbaci;
00021 using namespace nsbaci::types;
00022
00023 // guarantee: if ok == false, errors.size() > 0
00024 struct FileResult {
00025   FileResult() : ok(true) {}
00026   explicit FileResult(std::vector<Error> errs)
00027       : ok(errs.empty()), errors(std::move(errs)) {}
00028   explicit FileResult(Error error) : ok(false), errors({std::move(error)}) {}
00029
00030   FileResult(FileResult&&) noexcept = default;
00031   FileResult& operator=(FileResult&&) noexcept = default;
00032
00033   FileResult(const FileResult&) = default;
00034   FileResult& operator=(const FileResult&) = default;
00035
00036   bool ok;
00037   std::vector<Error> errors;
00038 };
00039
00040 struct saveResult : FileResult {
00041   saveResult() : FileResult() {}
00042   explicit saveResult(std::vector<Error> errs) : FileResult(std::move(errs)) {}
00043   explicit saveResult(Error error) : FileResult(std::move(error)) {}
00044
00045   saveResult(saveResult&&) noexcept = default;
00046   saveResult& operator=(saveResult&&) noexcept = default;
00047   saveResult(const saveResult&) = default;
00048   saveResult& operator=(const saveResult&) = default;
00049 };
00050
00051 struct LoadResult : FileResult {
00052   LoadResult() : FileResult() {}
00053   LoadResult(Text conts, File name)
00054       : FileResult(), contents(std::move(conts)), fileName(std::move(name)) {}
00055   explicit LoadResult(std::vector<Error> errs) : FileResult(std::move(errs)) {}
00056   explicit LoadResult(Error error) : FileResult(std::move(error)) {}
00057
00058   LoadResult(LoadResult&&) noexcept = default;
00059   LoadResult& operator=(LoadResult&&) noexcept = default;
00060   LoadResult(const LoadResult&) = default;
00061   LoadResult& operator=(const LoadResult&) = default;
00062
00063   Text contents;
00064   File fileName;
00065 };
00066
00071 namespace nsbaci::services {
00072
00076 class FileService {
00077  public:
00078   saveResult save(Text contents, File file);
00079
00080   LoadResult load(File file);
00081
00082   FileService() = default;
00083
00084   ~FileService() = default;
00085 };
00086
00087 }  // namespace nsbaci::services
00088
00089 #endif  // NSBACI_FILESERVICE_H
```

7.17 runtimeService.cpp File Reference

Implementation unit for the RuntimeService.

Include dependency graph for runtimeService.cpp:



Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.
- namespace nsbaci::services

  Services namespace for nsbaci.

7.17.1 Detailed Description

Implementation unit for the RuntimeService.

Author

Nicolás Serrano García

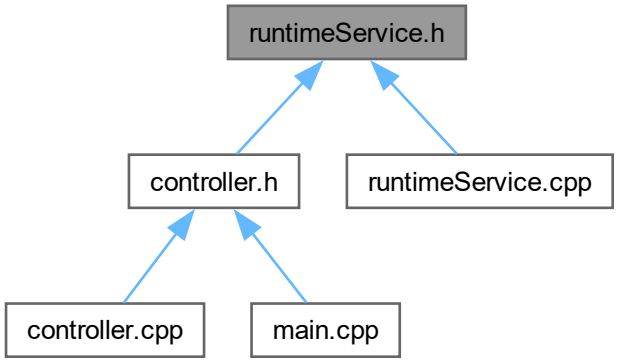
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.18 runtimeService.h File Reference

RuntimeService class declaration for nsbaci.

This graph shows which files directly or indirectly include this file:



Classes

- class nsbaci::services::RuntimeService

Namespaces

- namespace nsbaci::services

  Services namespace for nsbaci.
- namespace nsbaci

  Root namespace for the nsbaci application.

7.18.1 Detailed Description

RuntimeService class declaration for nsbaci.

This service is intended to encapsulate runtime-related utilities.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.19 runtimeService.h

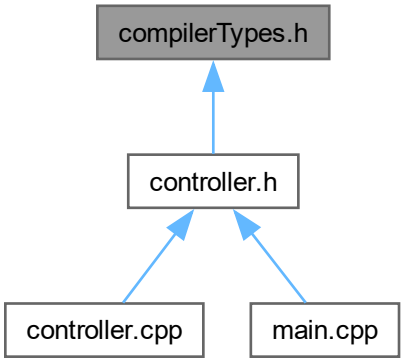
Go to the documentation of this file.

```
00001
00011
00012 #ifndef NSBACI_RUNTIMESERVICE_H
00013 #define NSBACI_RUNTIMESERVICE_H
00014
00019 namespace nsbaci::services {
00020
00024 class RuntimeService {
00025  public:
00026   RuntimeService() = default;
00027   ~RuntimeService() = default;
00028 };
00029
00030 }  // namespace nsbaci::services
00031
00032 #endif  // NSBACI_RUNTIMESERVICE_H
```

7.20 compilerTypes.h File Reference

Type definitions for compiler-related operations.

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace nsbaci::types

  Type definitions namespace for nsbaci.
- namespace nsbaci

  Root namespace for the nsbaci application.

7.20.1 Detailed Description

Type definitions for compiler-related operations.

This header provides type aliases used by the CompilerService and other components that work with compilation.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.21 compilerTypes.h

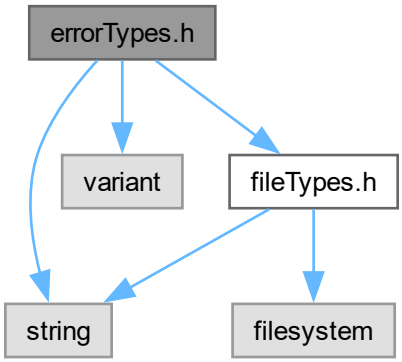
Go to the documentation of this file.

```
00001
00012
00013 #ifndef NSBACI_TYPES_COMPILERTYPES_H
00014 #define NSBACI_TYPES_COMPILERTYPES_H
00015
00020 namespace nsbaci::types {
00021
00022 // Compiler-specific types will be added here as needed
00023
00024 }  // namespace nsbaci::types
00025
00026 #endif  // NSBACI_TYPES_COMPILERTYPES_H
```

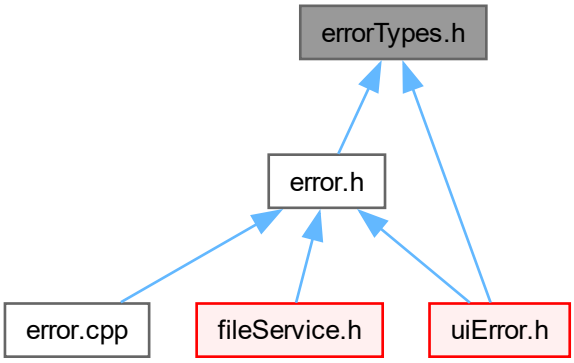
7.22 errorTypes.h File Reference

Type definitions for error-related structures.

Include dependency graph for errorTypes.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct nsbaci::types::ErrorBase

  Base structure containing common error properties.
- struct nsbaci::types::CompileError

  Error payload for compilation errors.
- struct nsbaci::types::SaveError

  Error payload for file save errors.
- struct nsbaci::types::LoadError

  Error payload for file load errors.
- struct nsbaci::types::RuntimeError

  Error payload for runtime errors.

Namespaces

- namespace nsbaci::types

  Type definitions namespace for nsbaci.
- namespace nsbaci

  Root namespace for the nsbaci application.

Typedefs

- using **nsbaci::types::ErrMessage** = std::string
- using nsbaci::types::ErrorPayload

  Variant type for all possible error payloads.

Enumerations

- enum class nsbaci::types::ErrSeverity { **Warning** , **Error** , **Fatal** }

  Severity levels for errors.
- enum class nsbaci::types::ErrType { **emptyPath** , **invalidPath** , **invalidExtension** , **directoryNotFound** , **fileNotFound** , **notARegularFile** , **permissionDenied** , **openFailed** , **readFailed** , **writeFailed** , **unknown** }

  Types of errors that can occur in the application.

7.22.1 Detailed Description

Type definitions for error-related structures.

This header provides type aliases and enums used by the Error module and other components that handle errors.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.23 errorTypes.h

Go to the documentation of this file.

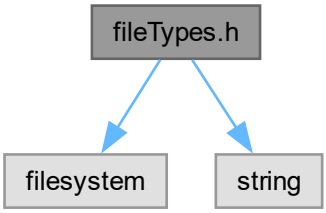
```
00001
00012
00013 #ifndef NSBACI_TYPES_ERRORTYPES_H
00014 #define NSBACI_TYPES_ERRORTYPES_H
00015
00016 #include <string>
00017 #include <variant>
00018
00019 #include "fileTypes.h"
00020
00025 namespace nsbaci::types {
00026
00031 enum class ErrSeverity { Warning, Error, Fatal };
00032
00037 enum class ErrType {
00038   // File path errors
00039   emptyPath,          // Path string is empty
00040   invalidPath,        // Path is malformed or invalid
00041   invalidExtension,   // File does not have .nsb extension
00042   directoryNotFound,  // Parent directory doesn't exist
00043   fileNotFound,       // File doesn't exist
00044   notARegularFile,    // Path points to directory, symlink, etc.
00045
00046   // Permission errors
00047   permissionDenied,  // No read/write access
00048
00049   // I/O errors
00050   openFailed,   // Could not open file
00051   readFailed,   // Error while reading
00052   writeFailed,  // Error while writing
00053
00054   // Generic
00055   unknown  // Unspecified error
00056 };
00057
00058 using ErrMessage = std::string;
00059
00064 struct ErrorBase {
```

```
  // this severity serves as an indicator to what icon to use in the error, the
  // string that appears in the top of the dialog, additional buttons for things
  // like restart when the error is fatal...
  ErrSeverity severity;
  ErrMessage message;
  // useful to show the reason for things. For example, if when trying to save a
  // file to a sensible location, if the app doesn't have privileges, the ui can
  // show a message of type: "/Error/ X could not be saved in Y. Reason:
  // Permission Denied. Try starting the app in admin mode. [Ok]"
  ErrType type;
};

struct CompileError {};

struct SaveError {
  File associatedFile;
};

struct LoadError {
  File associatedFile;
};

struct RuntimeError {};

using ErrorPayload =
    std::variant<SaveError, LoadError, CompileError, RuntimeError>;

}  // namespace nsbaci::types

#endif  // NSBACI_TYPES_ERRORTYPES_H
```

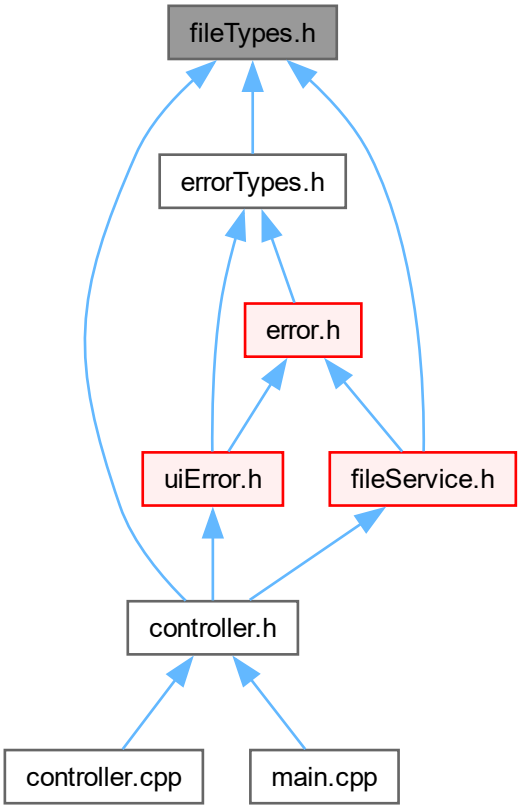
7.24 fileTypes.h File Reference

Type definitions for file-related operations.

Include dependency graph for fileTypes.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace nsbaci::types

  Type definitions namespace for nsbaci.
- namespace nsbaci

  Root namespace for the nsbaci application.

Typedefs

- using **nsbaci::types::Text** = std::string

  Alias for text content (file contents, source code, etc.).
- using **nsbaci::types::File** = fs::path

  Alias for file system paths.

7.24.1 Detailed Description

Type definitions for file-related operations.

This header provides type aliases used by the FileService and other components that work with files.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.25 fileTypes.h

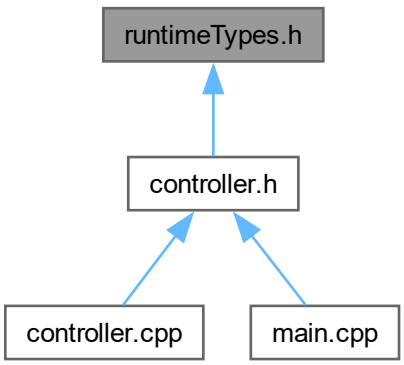
Go to the documentation of this file.

```
00001
00012
00013 #ifndef NSBACI_TYPES_FILETYPES_H
00014 #define NSBACI_TYPES_FILETYPES_H
00015
00016 #include <filesystem>
00017 #include <string>
00018
00023 namespace nsbaci::types {
00024
00025 namespace fs = std::filesystem;
00026
00030 using Text = std::string;
00031
00035 using File = fs::path;
00036
00037 }  // namespace nsbaci::types
00038
00039 #endif  // NSBACI_TYPES_FILETYPES_H
```

7.26 runtimeTypes.h File Reference

Type definitions for runtime-related operations.

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace nsbaci::types

  Type definitions namespace for nsbaci.
- namespace nsbaci

  Root namespace for the nsbaci application.

7.26.1 Detailed Description

Type definitions for runtime-related operations.

This header provides type aliases used by the RuntimeService and other components that work with runtime execution.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.27 runtimeTypes.h

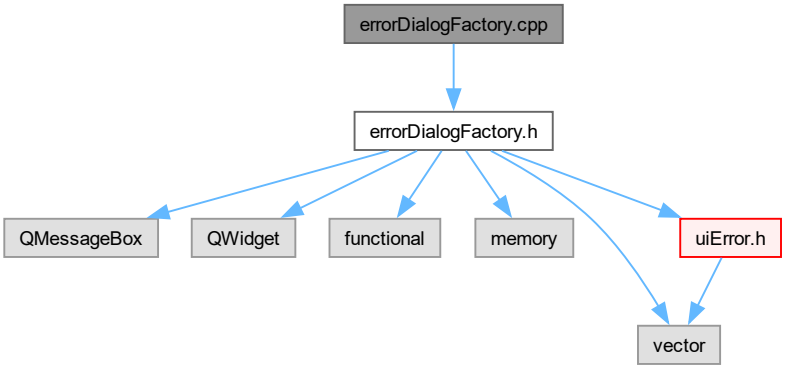
Go to the documentation of this file.

```
00001
00012
00013 #ifndef NSBACI_TYPES_RUNTIMETYPES_H
00014 #define NSBACI_TYPES_RUNTIMETYPES_H
00015
00020 namespace nsbaci::types {
00021
00022 // Runtime-specific types will be added here as needed
00023
00024 }  // namespace nsbaci::types
00025
00026 #endif  // NSBACI_TYPES_RUNTIMETYPES_H
```

7.28 errorDialogFactory.cpp File Reference

Implementation of ErrorDialogFactory.

Include dependency graph for errorDialogFactory.cpp:



Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.

7.28.1 Detailed Description

Implementation of ErrorDialogFactory.

Author

Nicolás Serrano García

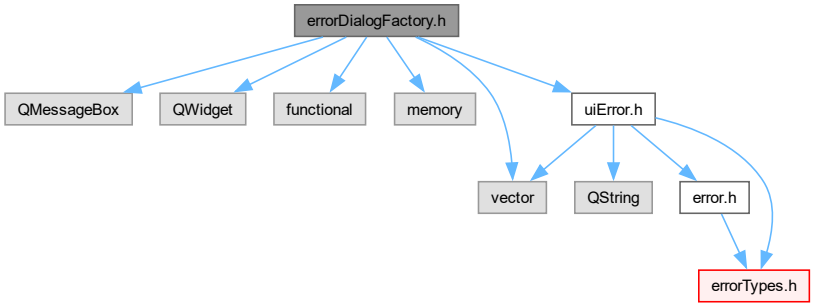
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

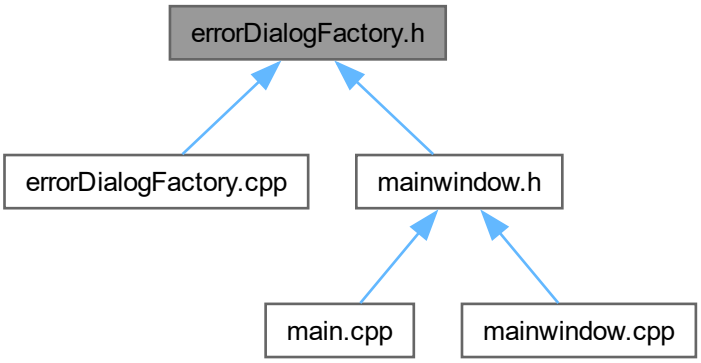
7.29 errorDialogFactory.h File Reference

Factory for creating error dialogs from UIError objects.

Include dependency graph for errorDialogFactory.h:



This graph shows which files directly or indirectly include this file:



Classes

- class nsbaci::ui::ErrorDialogFactory

  Factory for creating error dialogs from UIError objects.

Namespaces

- namespace nsbaci

  Root namespace for the nsbaci application.

7.29.1 Detailed Description

Factory for creating error dialogs from UIError objects.

This module provides a factory that converts UIError objects into ready-to-show QMessageBox dialogs. It is private to the UI layer.

7.29.1.1 Overview

The ErrorDialogFactory operates in two modes:

7.29.1.1.1 1. Deferred Mode (Factory Pattern)

Returns a **DialogInvoker** - a callable (`std::function`) that encapsulates all dialog data but does NOT show the dialog immediately. The caller decides when to invoke it. This is similar to a "lazy evaluation" or "thunk" pattern.

```
// Create the invoker (dialog is NOT shown yet)
auto dialogInvoker = ErrorDialogFactory::getDialogFromUIError(error, this);

// ... do other work, validation, logging, etc.

// Show the dialog when ready (blocks until user clicks)
QMessageBox::StandardButton clicked = dialogInvoker();

// React to user choice
if (clicked == QMessageBox::Close) {
    QApplication::quit();
}
```

7.29.1.1.2 2. Immediate Mode (Convenience)

Shows the dialog immediately and returns the result. Internally, this creates an invoker and calls it right away. Use this when you don't need to defer the dialog display.

```
// Show immediately - blocks until user clicks
ErrorDialogFactory::showError(error, this);

// Or capture the result
auto clicked = ErrorDialogFactory::showError(error, this);
```

7.29.1.2 Why Return Callables?

The DialogInvoker pattern provides:

- **Separation of concerns**: Factory knows HOW to build, caller knows WHEN to show
- **Deferred execution**: Prepare dialogs ahead of time, show when appropriate
- **User response handling**: The return value indicates which button was clicked
- **Flexibility**: Batch prepare multiple dialogs, show conditionally, etc.

The callable captures all necessary data by value, so the original UIError can be destroyed before the dialog is shown.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.30 errorDialogFactory.h

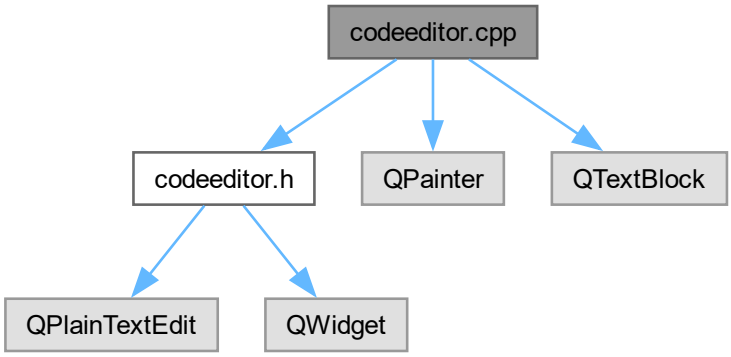
Go to the documentation of this file.

```
00001
00065
00066 #ifndef NSBACI_ERRORDIALOGFACTORY_H
00067 #define NSBACI_ERRORDIALOGFACTORY_H
00068
00069 #include <QMessageBox>
00070 #include <QWidget>
00071 #include <functional>
00072 #include <memory>
00073 #include <vector>
00074
00075 #include "uiError.h"
00076
00077 namespace nsbaci::ui {
00078
00090 class ErrorDialogFactory {
00091  public:
00102   using DialogInvoker = std::function<QMessageBox::StandardButton()>;
00103
00114   static DialogInvoker getDialogFromUIError(const UIError& error,
00115                                             QWidget* parent = nullptr);
00116
00124   static std::vector<DialogInvoker> getDialogsFromUIErrors(
00125       const std::vector<UIError>& errors, QWidget* parent = nullptr);
00126
00135   static DialogInvoker getSuccessDialog(const QString& title,
00136                                         const QString& message,
00137                                         QWidget* parent = nullptr);
00138
00148   static void showErrors(const std::vector<UIError>& errors,
00149                          QWidget* parent = nullptr);
00150
00158   static QMessageBox::StandardButton showError(const UIError& error,
00159                                                QWidget* parent = nullptr);
00160
00168   static void showSuccess(const QString& title, const QString& message,
00169                           QWidget* parent = nullptr);
00170
00171  private:
00175   static QMessageBox::Icon iconFromSeverity(ErrSeverity severity);
00176 };
00177
00178 }  // namespace nsbaci::ui
00179
00180 #endif  // NSBACI_ERRORDIALOGFACTORY_H
```

7.31 codeeditor.cpp File Reference

Implementation of the CodeEditor class with line numbers.

Include dependency graph for codeeditor.cpp:



7.31.1 Detailed Description

Implementation of the CodeEditor class with line numbers.

Author

Nicolás Serrano García

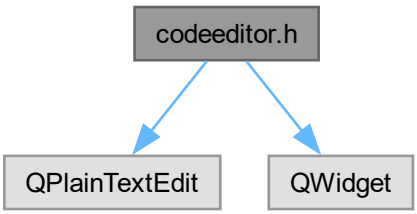
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

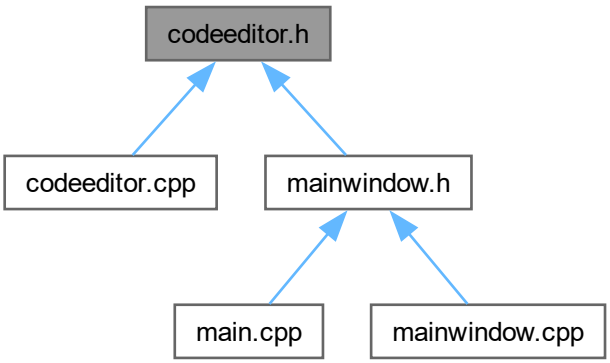
7.32 codeeditor.h File Reference

CodeEditor class with line numbers for nsbaci.

Include dependency graph for codeeditor.h:



This graph shows which files directly or indirectly include this file:



Classes

- class CodeEditor
- class LineNumberArea

7.32.1 Detailed Description

CodeEditor class with line numbers for nsbaci.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.33 codeeditor.h

Go to the documentation of this file.

```
00001
00009
00010 #ifndef CODEEDITOR_H
00011 #define CODEEDITOR_H
00012
00013 #include <QPlainTextEdit>
00014 #include <QWidget>
00015
00016 class LineNumberArea;
00017
00018 class CodeEditor : public QPlainTextEdit {
00019   Q_OBJECT
00020
00021  public:
00022   explicit CodeEditor(QWidget* parent = nullptr);
```

```
00023
00024   void lineNumberAreaPaintEvent(QPaintEvent* event);
00025   int lineNumberAreaWidth();
00026
00027  protected:
00028   void resizeEvent(QResizeEvent* event) override;
00029
00030  private slots:
00031   void updateLineNumberAreaWidth(int newBlockCount);
00032   void highlightCurrentLine();
00033   void updateLineNumberArea(const QRect& rect, int dy);
00034
00035  private:
00036   LineNumberArea* lineNumberArea;
00037 };
00038
00039 class LineNumberArea : public QWidget {
00040  public:
00041   explicit LineNumberArea(CodeEditor* editor)
00042       : QWidget(editor), codeEditor(editor) {}
00043
00044   QSize sizeHint() const override {
00045     return QSize(codeEditor->lineNumberAreaWidth(), 0);
00046   }
00047
00048  protected:
00049   void paintEvent(QPaintEvent* event) override {
00050     codeEditor->lineNumberAreaPaintEvent(event);
00051   }
00052
00053  private:
00054   CodeEditor* codeEditor;
00055 };
00056
00057 #endif  // CODEEDITOR_H
```

7.34 mainwindow.cpp File Reference

Implementation of the MainWindow class.

Include dependency graph for mainwindow.cpp:

7.34.1 Detailed Description

Implementation of the MainWindow class.

Author

Nicolás Serrano García

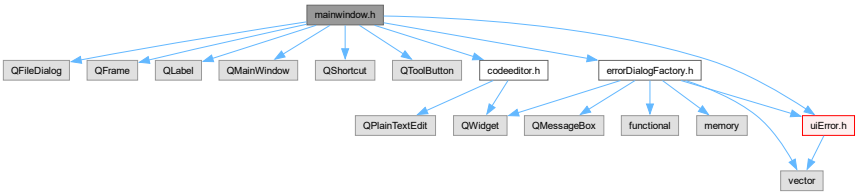
Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

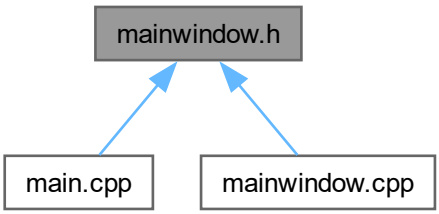
7.35 mainwindow.h File Reference

Main window class declaration for nsbaci.

Include dependency graph for mainwindow.h:



This graph shows which files directly or indirectly include this file:



Classes

- class MainWindow

7.35.1 Detailed Description

Main window class declaration for nsbaci.

Author

Nicolás Serrano García

Copyright

Copyright (c) 2025 Nicolás Serrano García. Licensed under the MIT License.

7.36 mainwindow.h

Go to the documentation of this file.

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QFileDialog>
#include <QFrame>
#include <QLabel>
#include <QMainWindow>
#include <QShortcut>
#include <QToolButton>

#include "codeeditor.h"
#include "errorDialogFactory.h"
#include "uiError.h"

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow {
  Q_OBJECT

 public:
  explicit MainWindow(QWidget* parent = nullptr);
  ~MainWindow() override = default;

 signals:
  void saveRequested(const QString& filePath, const QString& contents);
  void openRequested(const QString& filePath);
  void compileRequested(const QString& contents);
  void runRequested();

 public slots:
  void setEditorContents(const QString& contents);
  void setStatusMessage(const QString& message);
  void setCurrentFile(const QString& fileName, bool modified = false);

  // Controller response slots
  void onSaveSucceeded();
  void onSaveFailed(std::vector<nsbaci::UIError> errors);
  void onLoadSucceeded(const QString& contents);
  void onLoadFailed(std::vector<nsbaci::UIError> errors);

 private slots:
  // File menu
  void onNew();
  void onSave();
  void onSaveAs();
  void onOpen();
  void onExit();
  void onCompile();
  void onRun();

  // Edit menu
  void onUndo();
  void onRedo();
  void onCut();
  void onCopy();
  void onPaste();
  void onSelectAll();

  // View menu
  void onToggleSidebar();
  void onToggleFullscreen();

  // Help menu
  void onAbout();

  // Editor
  void onTextChanged();

 private:
  // File info bar
  QFrame* fileInfoBar = nullptr;
  QLabel* fileNameLabel = nullptr;
  QLabel* fileModifiedIndicator = nullptr;

  // Central widget
  CodeEditor* codeEditor = nullptr;
```

```

    // Sidebar
    QFrame* sideBar = nullptr;
    QToolButton* compileButton = nullptr;
    QToolButton* runButton = nullptr;

    // File actions
    QAction* actionNew = nullptr;
    QAction* actionSave = nullptr;
    QAction* actionSaveAs = nullptr;
    QAction* actionOpen = nullptr;
    QAction* actionExit = nullptr;

    // Edit actions
    QAction* actionUndo = nullptr;
    QAction* actionRedo = nullptr;
    QAction* actionCut = nullptr;
    QAction* actionCopy = nullptr;
    QAction* actionPaste = nullptr;
    QAction* actionSelectAll = nullptr;

    // View actions
    QAction* actionToggleSidebar = nullptr;
    QAction* actionFullscreen = nullptr;

    // Build actions
    QAction* actionCompile = nullptr;
    QAction* actionRun = nullptr;

    // Help actions
    QAction* actionAbout = nullptr;

    // State
    QString currentFileName;  // Just the filename for display
    QString currentFilePath;  // Full path for saving
    bool isModified = false;
    bool hasName = false;

   private:
    void createCentralWidget();
    void createMenuBar();
    void createStatusBar();
    void setupShortcuts();
    void applyStyleSheet();
};
#endif  // MAINWINDOW_H
```